CEBU EASTERN COLLEGE, INC.
College of Information Technology

LEARNING HANDOUT

Modular PHP Application Development

Course	Week	Term	School Year
IT WST 21 – Web Systems and Technologies 2	Week 8 (Midterm Period)	1st Semester	2026–2027

Learning Outcome

By the end of this lesson, you should be able to develop modular PHP applications using industry-standard coding practices and application organization techniques.

Up to now, most of our scripts have lived in a single file. Real applications quickly become too large and messy to manage that way. This week is about organizing PHP code the way professional developers do — splitting it into clear, reusable, well-structured pieces — and getting you ready for the MVC frameworks we'll use later in the course.

1. Organizing PHP Projects

A well-organized project makes it easy for you — and anyone else working on it — to find and update code quickly. Instead of dumping every file in one folder, group files by purpose.

A Typical Project Structure

```
my-project/
  config/
    database.php
  includes/
    header.php
    footer.php
    functions.php
  pages/
    index.php
    login.php
    dashboard.php
  assets/
    css/
    js/
    images/
```

- config/ – settings that rarely change, like database credentials.
- includes/ – reusable pieces shared across many pages (header, footer, helper functions).
- pages/ – the actual pages a visitor requests.
- assets/ – static files: stylesheets, JavaScript, images.

This kind of structure is a small step toward the folder conventions used by frameworks such as Laravel and CodeIgniter.

2. Reusable Functions and Includes

Instead of retyping the same PHP code on every page, move it into its own file and pull it in wherever it's needed.

include and require

```
// includes/functions.php
<?php
  function
formatCurrency($amount) {
    return "₱" .
number_format($amount, 2);
  }
?>
```

```
// pages/dashboard.php
<?php
  require
"../includes/functions.php";

  echo formatCurrency(1500);
// ₱1,500.00
?>
```

Statement	Behavior if the file is missing
include	Emits a warning; script continues running
require	Fatal error; script stops immediately
include_once / require_once	Same as above, but skips the file if it was already included

- Use require (or require_once) for files your script cannot run without, such as database connections or core functions.
- Reusing header.php and footer.php through include keeps your site's layout consistent across every page.

```
<?php require "includes/header.php"; ?>

<h1>Welcome to the Dashboard</h1>

<?php require "includes/footer.php"; ?>
```

3. Separation of Logic and Presentation

Mixing database queries, calculations, and HTML together in one file quickly becomes hard to read and hard to fix. Separating logic (what the code does) from presentation (how it looks) is one of the most important habits in professional PHP development.

Before: Logic and HTML Mixed Together

```
<?php
  $students = ["Juan", "Maria", "Pedro"];
  foreach ($students as $s) {
    if (strlen($s) > 4) {
```

```
      echo "<li><strong>$s</strong></li>";
    } else {
      echo "<li>$s</li>";
    }
  }
?>
```

After: Logic Prepared First, Then Displayed

```
<?php
  // --- Logic ---
  $students = ["Juan", "Maria", "Pedro"];
?>

<!-- --- Presentation --- -->
<ul>
  <?php foreach ($students as $s): ?>
    <li><?= htmlspecialchars($s) ?></li>
  <?php endforeach; ?>
</ul>
```

- The alternative syntax (: ... endforeach;) is commonly used in template-style files because it reads more like HTML.
- <?= ?> is shorthand for <?php echo ?>, handy for printing values inside markup.
- Keeping business logic in plain .php files and markup in template-style files makes both easier to maintain.

4. Preparing for MVC-Based Development

MVC (Model–View–Controller) is a pattern that formalizes the separation you just practiced. Frameworks like Laravel and CodeIgniter are built around it, so understanding the pattern now will make those frameworks much easier to learn.

Layer	Responsibility	Example
Model	Handles data and database logic	A Student class that fetches records from MySQL
View	Displays data as HTML to the user	A dashboard.php template that only outputs markup
Controller	Connects the two — receives requests, asks the Model for data, loads the View	A DashboardController that fetches students and passes them to the view

A Miniature MVC-Style Example

```
// model.php – handles the data
<?php
  function getStudents() {
    return ["Juan", "Maria", "Pedro"];
  }
?>
```

```
// controller.php – ties everything together
<?php
  require "model.php";
```

```
  $students = getStudents();
  require "view.php";
?>
```

```
<!-- view.php – displays the data -->
<ul>
  <?php foreach ($students as $s): ?>
    <li><?= htmlspecialchars($s) ?></li>
  <?php endforeach; ?>
</ul>
```

- The controller never contains HTML, and the view never contains data logic — each file has one clear job.
- This pattern scales far better than a single large file as an application grows.

5. Key Takeaways

- Organize a PHP project into folders by purpose (config, includes, pages, assets) instead of one flat folder.
- Use include/require to reuse functions and layout pieces (headers, footers) instead of duplicating code.
- Separate logic from presentation: prepare your data first, then output markup using it.
- MVC formalizes this separation into Model (data), View (display), and Controller (coordination) — the foundation for the frameworks we'll use next.
- Writing modular, well-organized code now will make the transition to frameworks much smoother.

6. Learning Resources / References

PHP Official Documentation — https://www.php.net/docs.php

PHP Manual — https://www.php.net/manual/en/

W3Schools PHP Tutorial — https://www.w3schools.com/php/

Nixon, R. (2021). Learning PHP, MySQL & JavaScript: With jQuery, CSS & HTML5 (6th ed.). O'Reilly Media.

7. Practice Exercises

Complete the following exercises to prepare for our laboratory exercises, coding activity, and the modular PHP application project:

1. Create a small project folder structure (config/, includes/, pages/, assets/) and move a database connection script into config/database.php.

2. Write a includes/functions.php file with at least two reusable functions, then require it from two different pages.

3. Build shared includes/header.php and includes/footer.php files and require them on three different pages so the layout stays consistent.

4. Take an existing script that mixes PHP logic and HTML, and refactor it to prepare the data first, then output markup using the alternative syntax (foreach/endforeach).

5. Build a miniature MVC example: a model.php that returns an array of products, a controller.php that loads the model and includes the view, and a view.php that displays the products as an HTML list.